sudden usage spike (because the marketing department launched a 25%-off campaign without telling you) or due to a third-party integration failing and impacting your application. The scope of components you own or can directly influence determines what you should be focusing on in terms of observability.

The bottom line is that you don't want to fly blind. What exactly this means in the context of cloud-native systems is what we will explore in this chapter in a hands-on manner. While it's important to see things in action, as we progress, we will also try to capture the gist of the concepts via more formal means, including definitions.

This book assumes you are familiar with cloud-native environments. In general, you would expect to find microservice architectures, a large number of relatively short-lived components working together to provide the functionality. This includes cloud provider services (I'm using AWS to demonstrate the ideas here); container technologies, including Docker and Kubernetes; and function-as-a-service (FaaS) offerings, especially AWS Lambda. In case you want to read up, here are some suggestions:

- *Kubernetes in Action, Second Edition*, by Marko Lukša (Manning, 2020)
- *AWS Lambda in Action* by Danilo Poccia (Manning, 2016)

Further, I recommend *Software Telemetry* by Jamie Riedesel (Manning, 2021), which is complementary to this book and provides useful deep dives into certain observability aspects we won't dive into in detail in this book.

In this book, we focus on cloud-native environments. We mainly use open source observability tooling so that you can try out everything without licensing costs. However, it is important to understand that while we use open source tooling to show the concepts in action, they are universally applicable. That is, in a professional environment, you should always consider offloading parts or all of the tooling to the managed offerings your cloud provider of choice has or, equally, the offerings of observability vendors such as Datadog, Splunk, New Relic, Honeycomb, or Dynatrace. Before we get into cloud-native environments and what observability means in that context, let's step back a bit and look at it from a conceptual level.

1.1 What is observability?

What is observability, and why should you care? When we say *observability*, we mean trying to understand the internal system state via measuring data available to the outside. Typically, we do this to act upon it.

Before we get to a more formal definition of observability, let's review a few core concepts we will be using throughout the book:

- *System*—Short for *system under observation* (SUO). This is the cloud-native platform (and applications running on it) you care about and are responsible for.
- *Signals*—Information observable from the outside of a system. There are different signal types (the most common are logs, metrics, and traces), and they are generated by sources. Chapter 2 covers the signal types in detail.